

DETAILED THEORETICAL REPORT

MODULE 1 – AUTOMATION WITH SELENIUM

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Section-wise Theoretical Report with Illustrative Code

1.1 Introduction and Different Flavors of Selenium

Selenium is an open-source ecosystem used to automate web browsers. It is widely used for functional testing, regression testing, smoke testing, cross-browser testing and repetitive browser-based workflows. Selenium does not test the internal business logic of an application directly; instead, it automates the browser in a way similar to a real user.

The main Selenium components covered in the curriculum are Selenium IDE, Selenium RC, Selenium WebDriver and Selenium Grid. Selenium IDE is primarily a browser-based record-and-playback tool. Selenium RC was an older architecture that used a server between the test program and browser and has been replaced by WebDriver. Selenium WebDriver provides direct programmatic browser control through browser-specific driver implementations. Selenium Grid is intended for distributed and parallel execution across different browser and operating-system combinations.

A Selenium automation flow normally contains four stages: start a browser session, navigate to the required page, identify and interact with elements, and close the session. The quality of the automation depends heavily on reliable locators, synchronization and maintainable test design.

Selenium WebDriver example

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://example.com")

print(driver.title)
driver.quit()
```

1.2 Selenium WebDriver – Introduction and Installation

Selenium 4 WebDriver is the central API for controlling a browser from Python. Its architecture follows a client-server communication model in which the Python Selenium client sends WebDriver commands and the browser driver communicates with the browser.

A typical Python setup requires Python 3, the Selenium package and a supported browser such as Chrome, Edge or Firefox. Current Selenium versions can use Selenium Manager to locate and manage compatible browser drivers in many environments. On Windows, the traditional approach was to place a driver executable on PATH; however, automatic driver management is generally simpler.

A project should use a virtual environment so that Selenium and related dependencies remain isolated from unrelated Python projects. The basic installation command is `pip install selenium`. The same test concept can be executed against Chrome, Edge or Firefox by changing the WebDriver class.

Installation

```
python -m pip install selenium
```

Running Chrome, Edge and Firefox

```
from selenium import webdriver

chrome = webdriver.Chrome()
chrome.get("https://example.com")
chrome.quit()

edge = webdriver.Edge()
edge.get("https://example.com")
edge.quit()

firefox = webdriver.Firefox()
firefox.get("https://example.com")
firefox.quit()
```

1.3 Locators and Object Identification

A locator is the mechanism used to identify a web element in the DOM. The curriculum covers ID, name, tag name, link text, class name, CSS selectors and XPath. A locator should be stable, unique and understandable.

By.ID is usually preferred when a stable unique id exists. By.NAME is useful for form fields with reliable name attributes. By.CLASS_NAME can identify an element using a single class token, while By.TAG_NAME is useful for collections such as all links or table cells. Link text and partial link text are appropriate for anchor elements. CSS selectors provide concise expressions and support attributes, descendants, child relationships and wildcards. XPath is more expressive when parent, sibling and text relationships are needed.

The difference between locating one element and a list of elements is important. `find_element()` returns one matching element and raises an exception if no match is found. `find_elements()` returns a list and returns an empty list when there are no matches.

Common locator examples

```
from selenium.webdriver.common.by import By

driver.find_element(By.ID, "username")
driver.find_element(By.NAME, "password")
driver.find_element(By.CLASS_NAME, "login-button")
driver.find_element(By.TAG_NAME, "button")
driver.find_element(By.LINK_TEXT, "Login")
driver.find_element(By.PARTIAL_LINK_TEXT, "Log")
driver.find_element(By.CSS_SELECTOR, "input[type='email']")
driver.find_element(By.XPATH, "//button[@type='submit']")

buttons = driver.find_elements(By.TAG_NAME, "button")
```

CSS and XPath examples

```
# CSS attribute selector
driver.find_element(By.CSS_SELECTOR, "input[name='email']")

# CSS child selector
driver.find_element(By.CSS_SELECTOR, "form > input")

# XPath contains
driver.find_element(By.XPATH, "//button[contains(@class, 'login')]")

# XPath starts-with
driver.find_element(By.XPATH, "//input[starts-with(@id, 'user')]")

# XPath parent/sibling relationship
driver.find_element(By.XPATH, "//label[text()='Email']/following-sibling::input")
```

1.4 Handling Different Controls on a Web Page

Web applications contain buttons, input fields, checkboxes, radio buttons, select boxes and other interactive controls. Selenium represents these controls using WebElement objects. Common operations include `click()`, `send_keys()`, `clear()`, `is_selected()`, `is_enabled()`, `is_displayed()`, `text` and `get_attribute()`.

XPath can be absolute or relative. An absolute XPath starts from the root of the document and is usually fragile because a small DOM change can invalidate it. A relative XPath normally starts with `//` and expresses the relevant element relationship, making it more maintainable.

For a normal HTML select element, Selenium provides the Select helper. For custom JavaScript dropdowns, the control may need ordinary click and text-based element selection rather than Select. The same principle applies to calendars and other widgets: first understand the DOM structure and then select the required elements with stable locators.

Controls and Select

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

driver.find_element(By.ID, "username").clear()
driver.find_element(By.ID, "username").send_keys("Saheli")
driver.find_element(By.ID, "submit").click()

checkbox = driver.find_element(By.ID, "terms")
if not checkbox.is_selected():
    checkbox.click()

radio = driver.find_element(By.ID, "option1")
if not radio.is_selected():
    radio.click()

select = Select(driver.find_element(By.ID, "country"))
select.select_by_visible_text("India")

print(checkbox.is_enabled())
print(checkbox.is_displayed())
print(driver.find_element(By.ID, "username").get_attribute("value"))
```

1.5 Handling Different Controls – Advanced

Advanced browser automation includes alert boxes, date pickers, file uploads, multiple windows or tabs, keyboard events, scrolling, drag-and-drop, iframes, enabled/disabled state and element properties.

JavaScript alerts are outside the normal page DOM and must be accessed through `driver.switch_to.alert`. A file upload normally uses an HTML input element of type file; Selenium can send the local file path directly to that element. Multiple windows are managed through window handles. An iframe creates a separate browsing context, so Selenium must switch into it before locating elements inside it.

For keyboard and mouse interactions, Selenium provides ActionChains. JavaScript execution is available through `execute_script()`, although standard WebDriver APIs should be preferred whenever they are sufficient.

Alerts

```
driver.find_element(By.ID, "alertButton").click()
alert = driver.switch_to.alert
print(alert.text)
alert.accept()

driver.find_element(By.ID, "confirmButton").click()
driver.switch_to.alert.dismiss()

driver.find_element(By.ID, "promptButton").click()
prompt = driver.switch_to.alert
prompt.send_keys("Saheli")
prompt.accept()
```

File upload, iframe and windows

```
# File upload
file_input = driver.find_element(By.CSS_SELECTOR, "input[type='file']")
file_input.send_keys(r"C:\Users\Saheli\Documents\sample.txt")

# Iframe
frame = driver.find_element(By.ID, "payment-frame")
driver.switch_to.frame(frame)
driver.find_element(By.ID, "card").send_keys("1234")
driver.switch_to.default_content()

# Multiple windows
main = driver.current_window_handle
driver.execute_script("window.open('https://example.com', '_blank');")
new_handle = [h for h in driver.window_handles if h != main][0]
driver.switch_to.window(new_handle)
print(driver.title)
driver.close()
driver.switch_to.window(main)
```

Keyboard and scrolling

```
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.keys import Keys

element = driver.find_element(By.ID, "search")
ActionChains(driver).click(element).key_down(Keys.CONTROL).send_keys("a").key_up(Keys.CONTROL).send_keys("Selenium").perform()

driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
```

1.6 Handling Dynamic Elements and Web Tables

Dynamic elements may be created or modified after the initial page load. Their identifiers can change, or their visibility can depend on asynchronous requests. Reliable automation therefore combines flexible locators with synchronization.

The curriculum specifically includes WebTable handling. A table can be represented as rows and cells in the DOM. Selenium can locate all rows, iterate through them, extract cell text and compare a target value. Once the required row is identified, the corresponding Status, Price or other column can be read.

When a table has pagination, virtual scrolling or dynamic loading, the automation may need to repeatedly inspect the current rows and move to the next page until the target record is found.

Web table traversal

```
from selenium.webdriver.common.by import By

rows = driver.find_elements(By.CSS_SELECTOR, "#table1 tbody tr")

target_name = "Doe"
result = None

for row in rows:
    cells = row.find_elements(By.TAG_NAME, "td")
    if cells and target_name in cells[0].text:
        result = cells[4].text
        break

assert result is not None
print("Matching value:", result)
```

1.7 Synchronization and Wait Types

Synchronization is necessary because a web page may not be ready when the test attempts to interact with an element. AJAX calls, animations, JavaScript rendering and network delays can all create timing differences.

An implicit wait is configured once and applies to element lookup throughout the WebDriver session. An explicit wait targets a particular condition and repeatedly checks until the condition is satisfied or the timeout expires. Expected conditions include presence, visibility, clickability, title and alert conditions.

Explicit waits are generally more precise for dynamic applications. Fixed delays such as `time.sleep(5)` do not know whether the element became ready after one second or needs longer than five seconds, so they can make tests unnecessarily slow or unreliable.

Implicit wait

```
driver.implicitly_wait(10)
element = driver.find_element(By.ID, "dynamic-element")
```

Explicit wait

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

wait = WebDriverWait(driver, 10)

element = wait.until(
    EC.visibility_of_element_located((By.ID, "dynamic-element"))
)

button = wait.until(
    EC.element_to_be_clickable((By.ID, "submit"))
)
button.click()
```

1.8 Screenshots and Exception Handling

Screenshots provide visual evidence of the browser state at the time of a failure. Selenium can capture the complete viewport using `save_screenshot()` or related screenshot APIs. In a real framework, screenshots are normally attached to test reports when a test fails.

Python exception handling uses `try`, `except` and `finally`. Selenium exceptions such as `NoSuchElementException`, `TimeoutException` and `StaleElementReferenceException` can be handled selectively. The `finally` block is important for cleanup because the browser should be closed even when a test fails.

A generic exception handler can be useful at framework boundaries, but test code should avoid hiding failures. After collecting diagnostic information, the exception should normally be re-raised so the test runner records the failure.

Screenshot with cleanup

```
from selenium import webdriver

driver = webdriver.Chrome()

try:
    driver.get("https://example.com")
    assert "Expected Title" in driver.title
except Exception as exc:
    driver.save_screenshot("failure.png")
    print("Failure:", exc)
    raise
finally:
    driver.quit()
```

1.9 Advanced Interactions and Test Data

Automation frameworks frequently need external test data. The curriculum covers Excel, JSON, properties, CSV and XML files, including reading and writing data. Externalizing data allows one test workflow to run against multiple input combinations without duplicating the automation logic.

Python's built-in `csv` and `json` modules cover common formats. Excel files can be handled with libraries such as `openpyxl` or `pandas`. XML can be parsed with Python's XML libraries. The exact library should be selected according to the data structure and project requirements.

Advanced interactions also include mouse hover and JavaScript commands. Selenium's ActionChains can move to an element before clicking or interacting with a hover menu. JavaScript can inspect or manipulate browser-side state, but it should not be used as a shortcut for normal WebDriver operations when a standard API is available.

JSON data

```
import json

with open("testdata.json", "r", encoding="utf-8") as f:
    data = json.load(f)

for case in data["login_cases"]:
    print(case["username"], case["expected"])
```

CSV data

```
import csv

with open("login_data.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["username"], row["password"])
```

Mouse hover and JavaScript

```
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By

menu = driver.find_element(By.ID, "products")
ActionChains(driver).move_to_element(menu).perform()

driver.execute_script(
    "arguments[0].scrollIntoView({block: 'center'});",
    menu
)
```

Module 1 – Summary

This module establishes the complete foundation of Selenium-based browser automation. It covers Selenium components, WebDriver installation, browser control, locator strategies, normal and advanced web controls, dynamic elements, synchronization, screenshots, exception handling, external test data, mouse interactions and JavaScript execution. The key engineering principle is to build automation that is stable, readable and maintainable rather than merely automating a sequence of clicks.

Key Learning Outcomes

- Explain Selenium IDE, RC, WebDriver and Grid.
- Configure Selenium WebDriver with Python and run tests on multiple browsers.
- Select robust elements using ID, name, CSS and XPath strategies.
- Automate buttons, inputs, checkboxes, radio buttons, select boxes and advanced browser contexts.
- Handle alerts, windows, tabs, iframes, file uploads and keyboard/mouse interactions.
- Traverse dynamic web tables and synchronize tests using waits.
- Capture screenshots and preserve failures using structured exception handling.
- Read and write external test data for data-driven automation.

Conclusion: A strong Selenium implementation combines correct element identification, proper synchronization, clean browser-context management and reusable test data handling. These concepts form the base for the subsequent PyTest, Page Object Model and broader automation framework topics.